Coinsult

Request your audit at coinsult.net

Advanced Manual
Smart Contract Audit

March 23, 2026

x.com/coinsultaudits

t.me/coinsult_tg

Audit requested by

Lifti Staking

0x08255708ce922df0043664babcaa3fde9b05d391

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
Informational	1	1	0	0
Low-Risk	2	2	0	0
Medium-Risk	4	4	0	0
Critical-Risk	1	1	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
Informational	Does not compromise the functionality of the contract in any way.
Low-Risk	Won't cause any problems, but can be adjusted for improvement.
Medium-Risk	Will likely cause problems and it is recommended to adjust.
Critical-Risk	Will definitely cause problems, this needs to be adjusted.

Audit Summary

Project	Lifti Staking
Website	https://www.lifti.io
Blockchain	Polygon
Source Code	https://polygonscan.com/address/0x08255708ce922df0043664bab-caa3fde9b05d391#code
Contract Address	0x08255708ce922df0043664babcaa3fde9b05d391

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code. Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

Slither: Solidity static analysis frameworkRemix: IDE Developer ToolCWE: Common Weakness EnumerationSWC: Smart Contract Weakness Classification and Test CasesDEX: Testnet Blockchains

Table of Contents

Global Overview 2

Audit Summary 3

Audit Scope 4

Detailed Findings

Admin can drain staked funds 6

Plan updates can break user expectations 7

Reward interval change affects existing stakes 8

Locked plan duration change has no effect 9

claimAll may run out of gas (DoS) 10

No validation that planId exists 11

reStake lock check is confusing 12

No emergency user withdrawal on pause 13

Disclaimer 14

Admin can drain staked funds

Critical-Risk *- Will definitely cause problems, this needs to be adjusted.*

Error Code	Description
AUD-001: Pending	The admin can call recoverERC20() to transfer any ERC20 (including the staking token) out of the contract. This can remove users' principal and/or rewards, making withdrawals fail and creating full custodial/centralization risk.

Code Snippet

```
function recoverERC20(address token,uint256 amount)
  external onlyRole(ADMIN_ROLE){
  IERC20(token).safeTransfer(msg.sender, amount);
}
```

Recommendation

Disallow recovering the stakingToken, or only allow recovering "excess" tokens above totalStakedPrincipal + owed rewards. Add timelock/multisig and emit clear events. Consider removing this function for production.

Plan updates can break user expectations

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-004: Pending	Admin can change lockDuration, rewardInterval, flexibility, min/max stake, and active status at any time. This can change how/when users can claim or restake and may effectively alter product terms after users deposit (centralization and fairness risk).

Code Snippet

```
plans[planId].lockDuration = newDuration;
plans[planId].rewardInterval = interval;
plans[planId].isFlexible = isFlexible;
plans[planId].isActive = active;
```

Recommendation

Freeze key parameters for existing stakes (store interval/duration/flexibility per stake) or apply changes only to new stakes. Add timelock and clear governance process for parameter changes.

Reward interval change affects existing stakes

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-005: Pending	pendingRewards() uses the current plan.rewardInterval, not a per-stake snapshot. If admin changes rewardInterval after staking, users' claim cadence and accrued rewards calculation can change unexpectedly, potentially reducing claimable rewards timing.

Code Snippet

```
uint256 intervals = elapsed / plan.rewardInterval;
... s.lastClaimTime = endTime - ((endTime - s.lastClaimTime) % plan.rewardInterval);
```

Recommendation

Store rewardInterval in StakeInfo at stake time (e.g., rewardIntervalAtStart) and use it for reward calculations and lastClaimTime rounding.

Locked plan duration change has no effect

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-006: Pending	lockEndTime is set at stake time and never updated. If admin updates plan.lockDuration later, it won't affect existing stakes, which may confuse users/admin and lead to incorrect assumptions about lock behavior.

Code Snippet

```
uint256 lockEnd = plan.isFlexible ? 0 : block.timestamp + plan.lockDuration;
... s.lockEndTime = lockEnd;
```

Recommendation

Either disallow changing lockDuration for active plans, or clearly document that changes apply only to new stakes. Consider storing duration per stake and exposing it in UI.

claimAll may run out of gas (DoS)

Medium-Risk *- Will likely cause problems and it is recommended to adjust.*

Error Code	Description
AUD-007: Pending	claimAll iterates over all userStakeIds. Users with many stakes may be unable to claim in one transaction due to gas limits, effectively preventing use of claimAll and potentially trapping rewards unless claimed individually.

Code Snippet

```
uint256[] memory ids = userStakeIds[msg.sender];
for (uint256 i = 0; i < ids.length; i++) { ... }
```

Recommendation

Add pagination (claimAll(uint256 start,uint256 count)) or allow batching by passing stakeIds. Keep claimRewards per stake as primary path.

No validation that planId exists

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-010: Pending	Functions read plans[planId] without checking existence. A non-created plan defaults to zeros; stake() will revert due to inactive, but admin update functions can silently modify a non-existent plan slot, creating confusing state.

Code Snippet

```
Plan memory plan = plans[planId];
plans[planId].aprBps = newAprBps;
```

Recommendation

Track plan existence (e.g., require(planId < nextPlanId)) in all plan admin/user functions. Consider an explicit 'exists' flag.

reStake lock check is confusing

Low-Risk *- Won't cause any problems, but can be adjusted for improvement.*

Error Code	Description
AUD-011: Pending	reStake requires plan.isFlexible OR lockEndTime >= now. This allows restaking only before lock expiry for locked plans, but the revert message says 'Lock expired'. Users may expect restake to extend lock or be allowed after expiry.

Code Snippet

```
require(plan.isFlexible || s.lockEndTime >= block.timestamp, "Lock expired");
```

Recommendation

Clarify intended behavior and rename message. If restake should extend lock, update lockEndTime accordingly; otherwise document that restake is only allowed during lock.

No emergency user withdrawal on pause

Informational *- Does not compromise the functionality of the contract in any way.*

Error Code	Description
AUD-013: Pending	whenNotPaused blocks stake/claim/unstake. If paused due to incident, users cannot withdraw principal, increasing trust requirements and potential loss if pause is prolonged.

Code Snippet

```
function unstake(...) external nonReentrant whenNotPaused { ... }
```

Recommendation

Allow an emergencyWithdrawPrincipal() callable when paused (optionally forfeiting rewards). Alternatively, allow unstake while paused and only block staking/claiming.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.